GOOGLE CLOUD SKILLS | HANDS-ON LAB REPORT

IAM Custom Roles

Least-privilege access with gcloud · Lab ID: GSP190

PREPARED BY	Yabesh J
DATE COMPLETED	20 October 2026
PLATFORM	Google Cloud (IAM, Cloud Shell)
LEVEL / DURATION	Introductory / 15 minutes
FINAL SCORE	100 / 100

1. Overview

Cloud IAM does not grant permissions to users directly. It grants **roles**, which bundle one or more permissions. **Predefined roles** are created and kept up to date by Google. **Custom roles** are defined by you from the available permissions, so you can give users and service accounts only what they need, which is the principle of least privilege. Google does not update custom roles automatically.

In this lab I used `gcloud iam` commands in Cloud Shell to create, list, update, disable, delete and restore project-level custom roles.

2. Objectives

- View the permissions available and the roles grantable on a resource
- Create custom roles using a YAML definition and using CLI flags
- Update existing custom roles with least-privilege permissions
- Disable, delete and restore custom roles

3. Concepts and Resources

Concept	Details
Permission	Written as *service.resource.verb*, for example compute.instances.list. Usually corresponds 1:1 with a REST method.
Custom role	A user-defined bundle of permissions at the project or organization level (not the folder level). Usable only within the project or organization that owns it.
Who can create one	A caller needs *iam.roles.create*: a project owner, or the IAM Role Administrator (roles/iam.roleAdmin) or Organization Role Administrator role.
etag	A version marker on a role. An update is written only if the etag matches the current one, which prevents concurrent edits from overwriting each other.
Launch stage	ALPHA, BETA, GA, or DISABLED. A DISABLED role's bindings are inactive.
Deletion	Existing bindings stay but become inactive. A role can be undeleted within 7 days. After 37 days its role ID can be reused.

Resource	Details
Project	qwiklabs-gcp-04-ea2634c7dcbe
Custom role “editor”	Title Role Editor, ALPHA. Permissions: appengine.versions.create, appengine.versions.delete, storage.buckets.get, storage.buckets.list
Custom role “viewer”	Title Role Viewer, ALPHA. Permissions: compute.instances.get, compute.instances.list, storage.buckets.get, storage.buckets.list

The project and roles were temporary and were removed when the lab ended.

4. Step-by-Step Walkthrough

Tasks 1–3 – Explore permissions and roles

I listed the permissions that can be applied to the project, described a role to see its metadata, and listed the roles that can be granted on the project:

```
gcloud iam list-testable-permissions \
  //cloudresourcemanager.googleapis.com/projects/$DEVSHELL_PROJECT_ID
gcloud iam roles describe roles/viewer
gcloud iam list-grantable-roles \
  //cloudresourcemanager.googleapis.com/projects/$DEVSHELL_PROJECT_ID
```

Task 4 – Create custom roles

I created the **editor** role from a YAML file and the **viewer** role with command-line flags, both at the project level:

```
# role-definition.yaml
title: "Role Editor"
description: "Edit access for App Versions"
stage: "ALPHA"
includedPermissions:
- appengine.versions.create
- appengine.versions.delete

gcloud iam roles create editor --project $DEVSHELL_PROJECT_ID --file role-definition.yaml

gcloud iam roles create viewer --project $DEVSHELL_PROJECT_ID \
  --title "Role Viewer" --description "Custom role description." \
  --permissions compute.instances.get,compute.instances.list --stage ALPHA
```

Task 5 – List custom roles

```
gcloud iam roles list --project $DEVSHELL_PROJECT_ID
```

Task 6 – Update custom roles

To update **editor** from a YAML file, I added storage.buckets.get and storage.buckets.list to *new-role-definition.yaml*. Because an update must carry the role's current etag, I read the latest etag with gcloud and wrote it into the file with `sed` before updating. The output lists the two storage permissions alongside the two App Engine ones, and a second **describe** confirms the change.

```
LATEST_ETAG=$(gcloud iam roles describe editor --project="$DEVSHELL_PROJECT_ID" --format="value(etag)")
sed -i "s/^etag:.*/etag: $LATEST_ETAG/" new-role-definition.yaml
gcloud iam roles update editor --project=$DEVSHELL_PROJECT_ID --file=new-role-definition.yaml
```

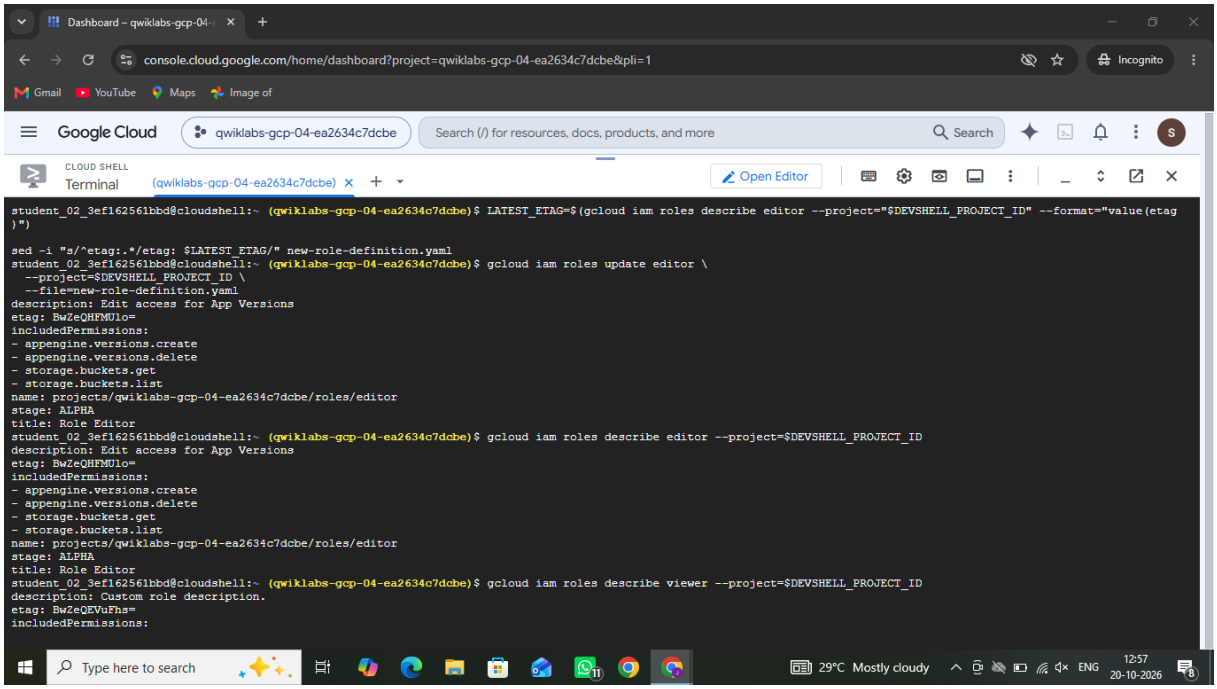


Figure 1: Latest etag read and written into the YAML file, editor role updated (four permissions, stage ALPHA), then verified with describe. The last command begins describing the viewer role.

Task 7 – Disable a custom role

Setting the stage to **DISABLED** inactivates any bindings to the role. I disabled the **viewer** role, confirmed the stage with describe, and then set its stage back to ALPHA so it could be used again.

```
gcloud iam roles update viewer --project=$DEVSHELL_PROJECT_ID --stage=DISABLED
gcloud iam roles update viewer --project=$DEVSHELL_PROJECT_ID --stage=ALPHA
```

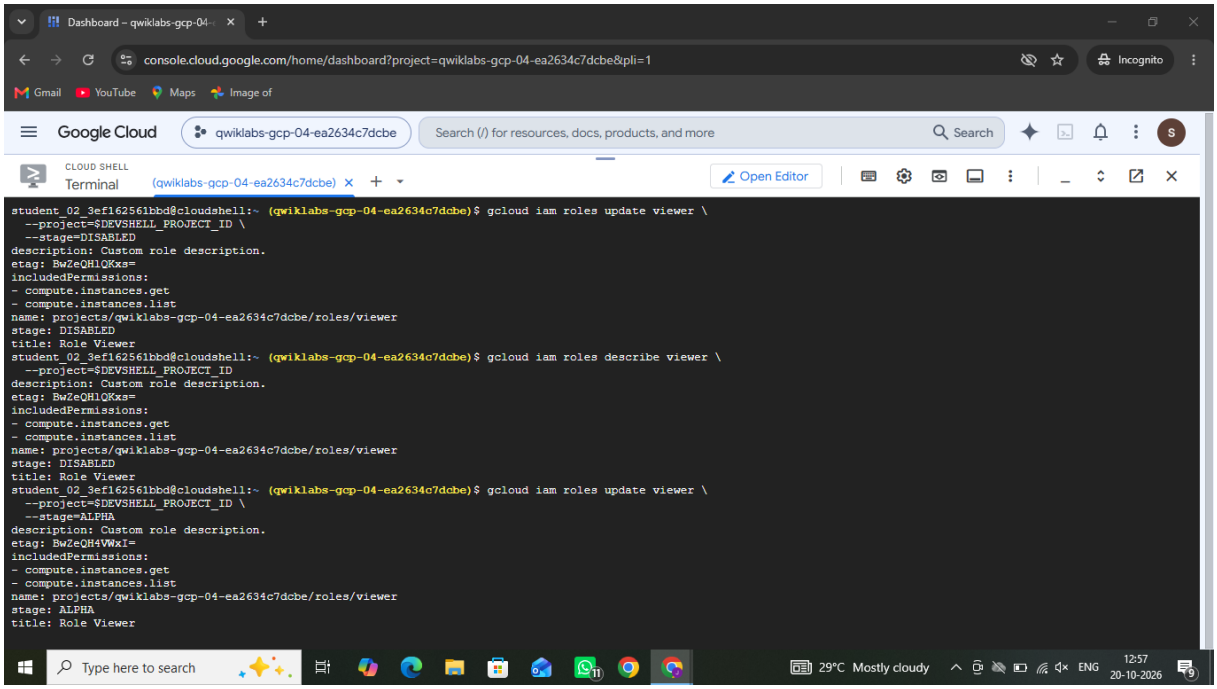


Figure 2: Viewer role set to DISABLED and confirmed with describe, then returned to ALPHA. At this point it holds only compute.instances.get and compute.instances.list.

Updating with flags, then deleting

I added two permissions to the **viewer** role with `--add-permissions`. The describe output shows all four permissions. Then I deleted the role with `gcloud iam roles delete`, which returns *deleted: true*.

```
gcloud iam roles update viewer --project=$DEVSHELL_PROJECT_ID \
  --add-permissions=storage.buckets.get,storage.buckets.list
gcloud iam roles delete viewer --project $DEVSHELL_PROJECT_ID
```

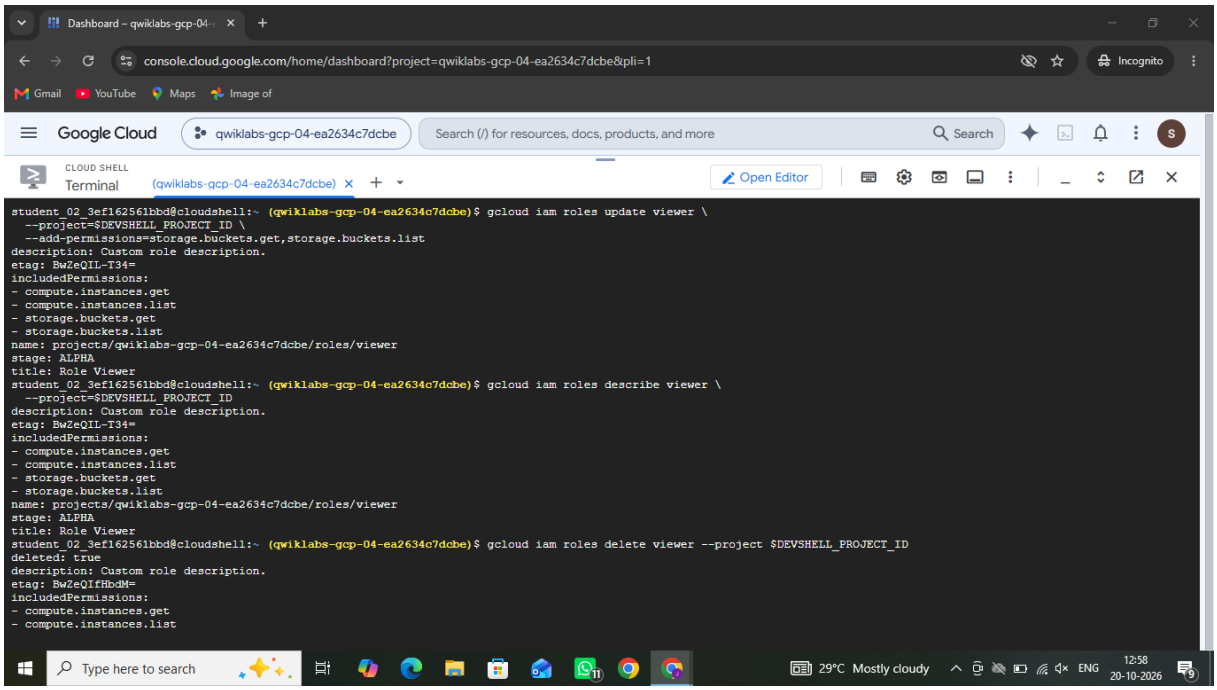


Figure 3: Permissions added to the viewer role (four in total), verified with describe, then the role deleted (deleted: true).

Task 9 – Restore a custom role

Within the 7-day window a deleted role can be restored with `gcloud iam roles undelete`. I restored the viewer role, deleted it again, and restored it a second time.

```
gcloud iam roles undelete viewer --project $DEVSHELL_PROJECT_ID
```

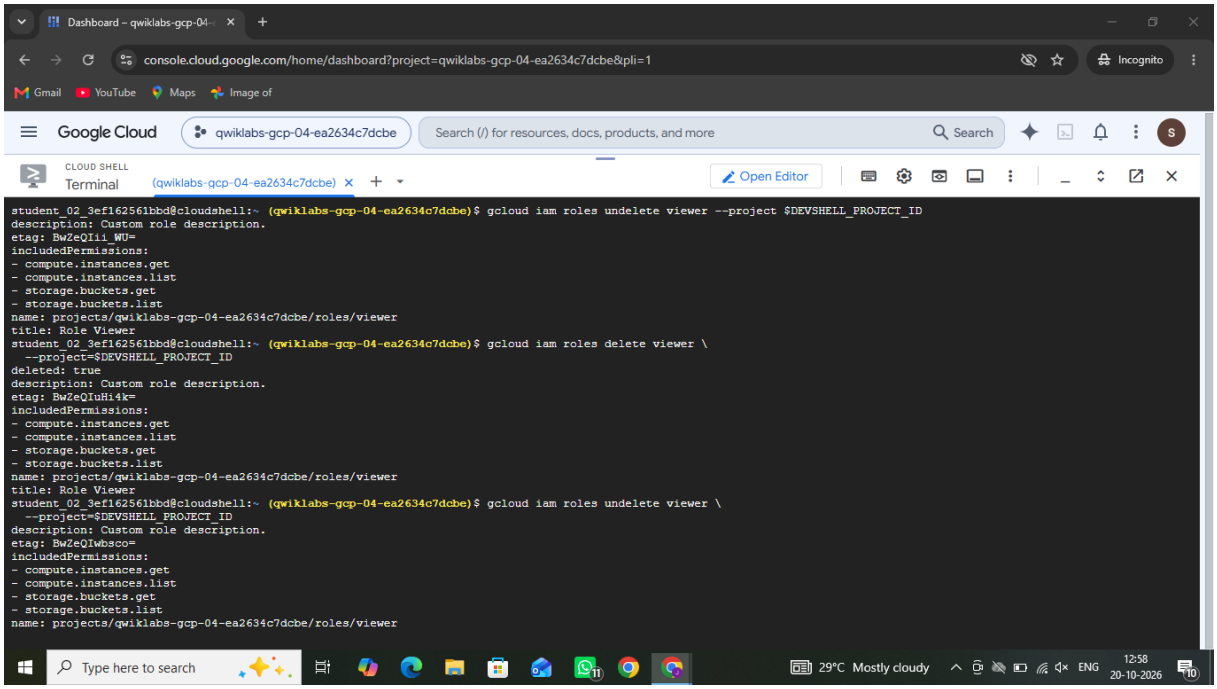


Figure 4: Viewer role undeleted, deleted again (deleted: true), and undeleted again. The restored role keeps all four permissions.

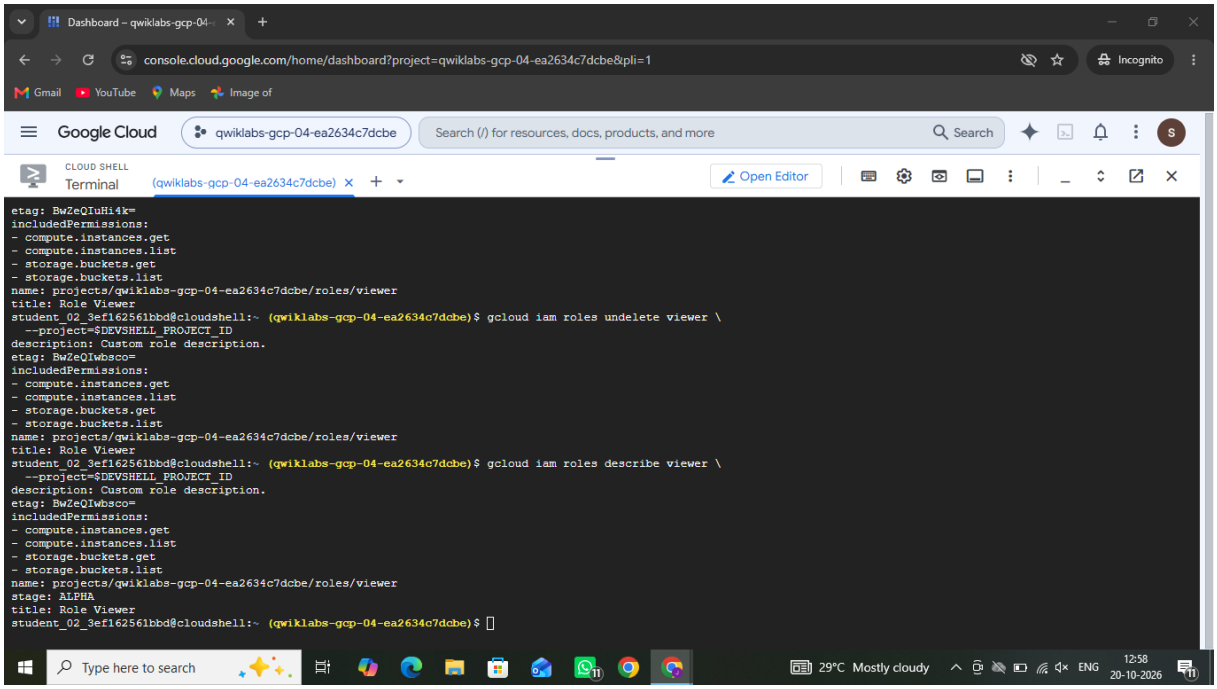


Figure 5: After the final undelete, describe shows the viewer role is back: stage ALPHA with compute.instances.get/list and storage.buckets.get/list.

5. Lab Completion

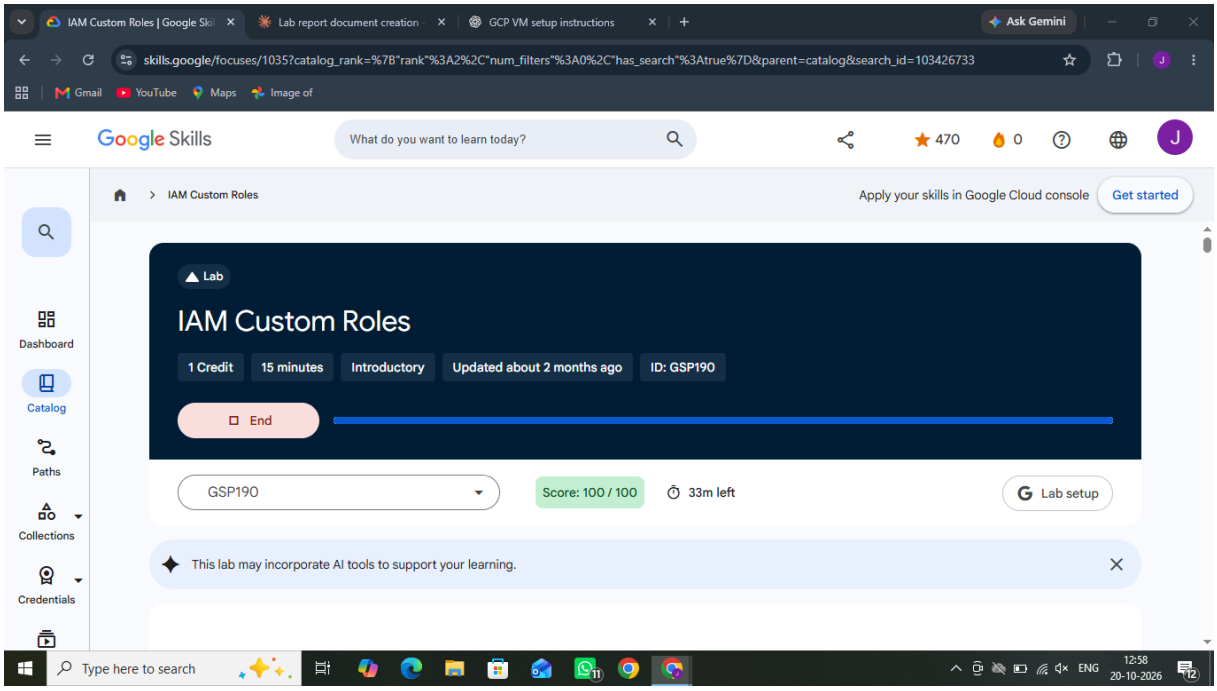


Figure 6: Lab completed with a score of 100/100.

6. Key Takeaways

- Roles bundle permissions. Custom roles let you choose exactly which ones, so users get only what they need (least privilege).
- A custom role can be created from a **YAML file** or with **flags**, at the project or organization level.
- Always use the role's current **etag** when updating, so that concurrent changes are not overwritten.
- `--add-permissions` and `--remove-permissions` change a role without rewriting its whole definition.
- Setting the stage to **DISABLED** makes the role's bindings inactive without deleting it.
- A deleted role keeps its bindings (inactive) and can be **undeleted within 7 days**. Its role ID can be reused after 37 days.
- Custom roles are not updated by Google, so new permissions must be added to them by hand.

7. Conclusion

The lab was completed with a score of 100/100. I created custom roles from a YAML file and from flags, updated them with new permissions, disabled a role, and deleted and restored it, which covers the full lifecycle of a custom IAM role.